JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time

Class OffsetTime

java.lang.Object
    java.time.OffsetTime

All Implemented Interfaces:

`Serializable`, `Comparable<OffsetTime>`, `Temporal`, `TemporalAccessor`, `TemporalAdjuster`

```
public final class OffsetTime
extends Object
implements Temporal, TemporalAdjuster, Comparable<OffsetTime>, Serializable
```

A time with an offset from UTC/Greenwich in the ISO-8601 calendar system, such as `10:15:30+01:00`.

`OffsetTime` is an immutable date-time object that represents a time, often viewed as hour-minute-second-offset. This class stores all time fields, to a precision of nanoseconds, as well as a zone offset. For example, the value "13:45.30.123456789+02:00" can be stored in an `OffsetTime`.

This is a value-based class; use of identity-sensitive operations (including reference equality (==), identity hash code, or synchronization) on instances of `OffsetTime` may have unpredictable results and should be avoided. The `equals` method should be used for comparisons.

Implementation Requirements:

`This class is immutable and thread-safe.`

Since:

`1.8`

See Also:

`Serialized Form`

Field Summary

Fields

Modifier and Type	Field	Description
static OffsetTime	MAX	The maximum supported OffsetTime, '23:59:59.999999999-18:00'.
static OffsetTime	MIN	The minimum supported OffsetTime, '00:00:00+18:00'.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
Temporal	adjustInto(Temporal temporal)	Adjusts the specified temporal object to have the same offset and time as this object.
OffsetDateTime	atDate(LocalDate date)	Combines this time with a date to create an OffsetDateTime.
int	compareTo(OffsetTime other)	Compares this OffsetTime to another time.
boolean	equals(Object obj)	Checks if this time is equal to another time.
String	format(DateTimeFormatter formatter)	Formats this time using the specified formatter.
static OffsetTime	from(TemporalAccessor temporal)	Obtains an instance of OffsetTime from

		a temporal object.
int	**get**(**TemporalField** field)	Gets the value of the specified field from this time as an `int`.
int	**getHour**()	Gets the hour-of-day field.
long	**getLong**(**TemporalField** field)	Gets the value of the specified field from this time as a `long`.
int	**getMinute**()	Gets the minute-of-hour field.
int	**getNano**()	Gets the nano-of-second field.
ZoneOffset	**getOffset**()	Gets the zone offset, such as '+01:00'.
int	**getSecond**()	Gets the second-of-minute field.
int	**hashCode**()	A hash code for this time.
boolean	**isAfter**(**OffsetTime** other)	Checks if the instant of this `OffsetTime` is after that of the specified time applying both times to a common date.
boolean	**isBefore**(**OffsetTime** other)	Checks if the instant of this `OffsetTime` is before that of the specified time applying both times to a common date.

boolean	**isEqual**(**OffsetTime** other)	Checks if the instant of this `OffsetTime` is equal to that of the specified time applying both times to a common date.
boolean	**isSupported**(**TemporalField** field)	Checks if the specified field is supported.
boolean	**isSupported**(**TemporalUnit** unit)	Checks if the specified unit is supported.
OffsetTime	**minus**(long amountToSubtract, **TemporalUnit** unit)	Returns a copy of this time with the specified amount subtracted.
OffsetTime	**minus**(**TemporalAmount** amountToSubtract)	Returns a copy of this time with the specified amount subtracted.
OffsetTime	**minusHours**(long hours)	Returns a copy of this `OffsetTime` with the specified number of hours subtracted.
OffsetTime	**minusMinutes**(long minutes)	Returns a copy of this `OffsetTime` with the specified number of minutes subtracted.
OffsetTime	**minusNanos**(long nanos)	Returns a copy of this `OffsetTime` with the specified number of

		nanoseconds subtracted.
`OffsetTime`	`minusSeconds(long seconds)`	Returns a copy of this `OffsetTime` with the specified number of seconds subtracted.
static `OffsetTime`	`now()`	Obtains the current time from the system clock in the default time-zone.
static `OffsetTime`	`now(Clock clock)`	Obtains the current time from the specified clock.
static `OffsetTime`	`now(ZoneId zone)`	Obtains the current time from the system clock in the specified time-zone.
static `OffsetTime`	`of(int hour, int minute, int second, int nanoOfSecond, ZoneOffset offset)`	Obtains an instance of `OffsetTime` from an hour, minute, second and nanosecond.
static `OffsetTime`	`of(LocalTime time, ZoneOffset offset)`	Obtains an instance of `OffsetTime` from a local time and an offset.
static `OffsetTime`	`ofInstant(Instant instant, ZoneId zone)`	Obtains an instance of `OffsetTime` from an `Instant` and zone ID.
static `OffsetTime`	`parse(CharSequence text)`	

		Obtains an instance of `OffsetTime` from a text string such as `10:15:30+01:00`.
static **OffsetTime**	**parse**(**CharSequence** text, **DateTimeFormatter** formatter)	Obtains an instance of `OffsetTime` from a text string using a specific formatter.
OffsetTime	**plus**(long amountToAdd, **TemporalUnit** unit)	Returns a copy of this time with the specified amount added.
OffsetTime	**plus**(**TemporalAmount** amountToAdd)	Returns a copy of this time with the specified amount added.
OffsetTime	**plusHours**(long hours)	Returns a copy of this `OffsetTime` with the specified number of hours added.
OffsetTime	**plusMinutes**(long minutes)	Returns a copy of this `OffsetTime` with the specified number of minutes added.
OffsetTime	**plusNanos**(long nanos)	Returns a copy of this `OffsetTime` with the specified number of nanoseconds added.
OffsetTime	**plusSeconds**(long seconds)	Returns a copy of this `OffsetTime` with the specified

		number of seconds added.
<R> R	**query**(**TemporalQuery**<R> query)	Queries this time using the specified query.
ValueRange	**range**(**TemporalField** field)	Gets the range of valid values for the specified field.
LocalTime	**toLocalTime**()	Gets the `LocalTime` part of this date-time.
String	**toString**()	Outputs this time as a `String`, such as `10:15:30+01:00`.
OffsetTime	**truncatedTo**(**TemporalUnit** unit)	Returns a copy of this `OffsetTime` with the time truncated.
long	**until**(**Temporal** endExclusive, **TemporalUnit** unit)	Calculates the amount of time until another time in terms of the specified unit.
OffsetTime	**with**(**TemporalAdjuster** adjuster)	Returns an adjusted copy of this time.
OffsetTime	**with**(**TemporalField** field, long newValue)	Returns a copy of this time with the specified field set to a new value.
OffsetTime	**withHour**(int hour)	Returns a copy of this `OffsetTime` with the hour-of-day altered.

OffsetTime	withMinute(int minute)	Returns a copy of this OffsetTime with the minute-of-hour altered.
OffsetTime	withNano(int nanoOfSecond)	Returns a copy of this OffsetTime with the nano-of-second altered.
OffsetTime	withOffsetSameInstant(ZoneOffset offset)	Returns a copy of this OffsetTime with the specified offset ensuring that the result is at the same instant on an implied day.
OffsetTime	withOffsetSameLocal(ZoneOffset offset)	Returns a copy of this OffsetTime with the specified offset ensuring that the result has the same local time.
OffsetTime	withSecond(int second)	Returns a copy of this OffsetTime with the second-of-minute altered.

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Field Detail

MIN

```
public static final OffsetTime MIN
```

The minimum supported `OffsetTime`, '00:00:00+18:00'. This is the time of midnight at the start of the day in the maximum offset (larger offsets are earlier on the time-line). This combines `LocalTime.MIN` and `ZoneOffset.MAX`. This could be used by an application as a "far past" date.

MAX

```
public static final OffsetTime MAX
```

The maximum supported `OffsetTime`, '23:59:59.999999999-18:00'. This is the time just before midnight at the end of the day in the minimum offset (larger negative offsets are later on the time-line). This combines `LocalTime.MAX` and `ZoneOffset.MIN`. This could be used by an application as a "far future" date.

Method Detail

now

```
public static OffsetTime now()
```

Obtains the current time from the system clock in the default time-zone.

This will query the `system clock` in the default time-zone to obtain the current time. The offset will be calculated from the time-zone in the clock.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Returns:

`the current time using the system clock and default time-zone, not null`

now

```
public static OffsetTime now(ZoneId zone)
```

Obtains the current time from the system clock in the specified time-zone.

This will query the system clock to obtain the current time. Specifying the time-zone avoids dependence on the default time-zone. The offset will be calculated from the specified time-zone.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Parameters:

`zone - the zone ID to use, not null`

Returns:

`the current time using the system clock, not null`

now

```
public static OffsetTime now(Clock clock)
```

Obtains the current time from the specified clock.

This will query the specified clock to obtain the current time. The offset will be calculated from the time-zone in the clock.

Using this method allows the use of an alternate clock for testing. The alternate clock may be introduced using dependency injection.

Parameters:

`clock - the clock to use, not null`

Returns:

`the current time, not null`

of

```
public static OffsetTime of(LocalTime time,
                            ZoneOffset offset)
```

Obtains an instance of `OffsetTime` from a local time and an offset.

Parameters:

`time - the local time, not null`

`offset - the zone offset, not null`

Returns:

`the offset time, not null`

of

```
public static OffsetTime of(int hour,
                            int minute,
                            int second,
                            int nanoOfSecond,
                            ZoneOffset offset)
```

Obtains an instance of `OffsetTime` from an hour, minute, second and nanosecond.

This creates an offset time with the four specified fields.

This method exists primarily for writing test cases. Non test-code will typically use other methods to create an offset time. `LocalTime` has two additional convenience variants of the equivalent factory method taking fewer arguments. They are not provided here to reduce the footprint of the API.

Parameters:

`hour` - the hour-of-day to represent, from 0 to 23

`minute` - the minute-of-hour to represent, from 0 to 59

`second` - the second-of-minute to represent, from 0 to 59

`nanoOfSecond` - the nano-of-second to represent, from 0 to 999,999,999

`offset` - the zone offset, not null

Returns:

the offset time, not null

Throws:

`DateTimeException` - if the value of any field is out of range

ofInstant

```
public static OffsetTime ofInstant(Instant instant,
                                   ZoneId zone)
```

Obtains an instance of `OffsetTime` from an `Instant` and zone ID.

This creates an offset time with the same instant as that specified. Finding the offset from UTC/Greenwich is simple as there is only one valid offset for each instant.

The date component of the instant is dropped during the conversion. This means that the conversion can never fail due to the instant being out of the valid range of dates.

Parameters:

`instant - the instant to create the time from, not null`

`zone - the time-zone, which may be an offset, not null`

Returns:

`the offset time, not null`

from

`public static OffsetTime from(TemporalAccessor temporal)`

Obtains an instance of `OffsetTime` from a temporal object.

This obtains an offset time based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `OffsetTime`.

The conversion extracts and combines the `ZoneOffset` and the `LocalTime` from the temporal object. Implementations are permitted to perform optimizations such as accessing those fields that are equivalent to the relevant objects.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `OffsetTime::from`.

Parameters:

`temporal - the temporal object to convert, not null`

Returns:

`the offset time, not null`

Throws:

`DateTimeException - if unable to convert to an OffsetTime`

parse

`public static OffsetTime parse(CharSequence text)`

Obtains an instance of `OffsetTime` from a text string such as `10:15:30+01:00`.

The string must represent a valid time and is parsed using `DateTimeFormatter.ISO_OFFSET_TIME`.

Parameters:

`text - the text to parse such as "10:15:30+01:00", not null`

Returns:

the parsed local time, not null

Throws:

DateTimeParseException - if the text cannot be parsed

parse

```
public static OffsetTime parse(CharSequence text,
                               DateTimeFormatter formatter)
```

Obtains an instance of OffsetTime from a text string using a specific formatter.

The text is parsed using the formatter, returning a time.

Parameters:

text - the text to parse, not null

formatter - the formatter to use, not null

Returns:

the parsed offset time, not null

Throws:

DateTimeParseException - if the text cannot be parsed

isSupported

```
public boolean isSupported(TemporalField field)
```

Checks if the specified field is supported.

This checks if this time can be queried for the specified field. If false, then calling the range, get and with(TemporalField, long) methods will throw an exception.

If the field is a ChronoField then the query is implemented here. The supported fields are:

- NANO_OF_SECOND
- NANO_OF_DAY
- MICRO_OF_SECOND
- MICRO_OF_DAY
- MILLI_OF_SECOND
- MILLI_OF_DAY
- SECOND_OF_MINUTE
- SECOND_OF_DAY
- MINUTE_OF_HOUR
- MINUTE_OF_DAY
- HOUR_OF_AMPM

- CLOCK_HOUR_OF_AMPM
- HOUR_OF_DAY
- CLOCK_HOUR_OF_DAY
- AMPM_OF_DAY
- OFFSET_SECONDS

All other ChronoField instances will return false.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.isSupportedBy(TemporalAccessor) passing this as the argument. Whether the field is supported is determined by the field.

Specified by:

isSupported in interface TemporalAccessor

Parameters:

field - the field to check, null returns false

Returns:

true if the field is supported on this time, false if not

isSupported

```
public boolean isSupported(TemporalUnit unit)
```

Checks if the specified unit is supported.

This checks if the specified unit can be added to, or subtracted from, this offset-time. If false, then calling the plus(long, TemporalUnit) and minus methods will throw an exception.

If the unit is a ChronoUnit then the query is implemented here. The supported units are:

- NANOS
- MICROS
- MILLIS
- SECONDS
- MINUTES
- HOURS
- HALF_DAYS

All other ChronoUnit instances will return false.

If the unit is not a ChronoUnit, then the result of this method is obtained by invoking TemporalUnit.isSupportedBy(Temporal) passing this as the argument. Whether the unit is supported is determined by the unit.

Specified by:

isSupported in interface Temporal

Parameters:

unit - the unit to check, null returns false

Returns:

`true if the unit can be added/subtracted, false if not`

range

```
public ValueRange range(TemporalField field)
```

Gets the range of valid values for the specified field.

The range object expresses the minimum and maximum valid values for a field. This time is used to enhance the accuracy of the returned range. If it is not possible to return the range, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return appropriate range instances. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.rangeRefinedBy(TemporalAccessor)` passing `this` as the argument. Whether the range can be obtained is determined by the field.

Specified by:

`range in interface TemporalAccessor`

Parameters:

`field - the field to query the range for, not null`

Returns:

`the range of valid values for the field, not null`

Throws:

`DateTimeException - if the range for the field cannot be obtained`

`UnsupportedTemporalTypeException - if the field is not supported`

get

```
public int get(TemporalField field)
```

Gets the value of the specified field from this time as an `int`.

This queries this time for the value of the specified field. The returned value will always be within the valid range of values for the field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return valid values based on this time, except

NANO_OF_DAY and MICRO_OF_DAY which are too large to fit in an int and throw a DateTimeException. All other ChronoField instances will throw an UnsupportedTemporalTypeException.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.getFrom(TemporalAccessor) passing this as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

get in interface TemporalAccessor

Parameters:

field - the field to get, not null

Returns:

the value for the field

Throws:

DateTimeException - if a value for the field cannot be obtained or the value is outside the range of valid values for the field

UnsupportedTemporalTypeException - if the field is not supported or the range of values exceeds an int

ArithmeticException - if numeric overflow occurs

getLong

public long getLong(TemporalField field)

Gets the value of the specified field from this time as a long.

This queries this time for the value of the specified field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a ChronoField then the query is implemented here. The supported fields will return valid values based on this time. All other ChronoField instances will throw an UnsupportedTemporalTypeException.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.getFrom(TemporalAccessor) passing this as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

getLong in interface TemporalAccessor

Parameters:

field - the field to get, not null

Returns:

the value for the field

Throws:

DateTimeException - if a value for the field cannot be obtained

UnsupportedTemporalTypeException - if the field is not supported

ArithmeticException - if numeric overflow occurs

getOffset

public ZoneOffset getOffset()

Gets the zone offset, such as '+01:00'.

This is the offset of the local time from UTC/Greenwich.

Returns:

the zone offset, not null

withOffsetSameLocal

public OffsetTime withOffsetSameLocal(ZoneOffset offset)

Returns a copy of this OffsetTime with the specified offset ensuring that the result has the same local time.

This method returns an object with the same LocalTime and the specified ZoneOffset. No calculation is needed or performed. For example, if this time represents 10:30+02:00 and the offset specified is +03:00, then this method will return 10:30+03:00.

To take into account the difference between the offsets, and adjust the time fields, use withOffsetSameInstant(java.time.ZoneOffset).

This instance is immutable and unaffected by this method call.

Parameters:

offset - the zone offset to change to, not null

Returns:

an OffsetTime based on this time with the requested offset, not null

withOffsetSameInstant

public OffsetTime withOffsetSameInstant(ZoneOffset offset)

Returns a copy of this `OffsetTime` with the specified offset ensuring that the result is at the same instant on an implied day.

This method returns an object with the specified `ZoneOffset` and a `LocalTime` adjusted by the difference between the two offsets. This will result in the old and new objects representing the same instant on an implied day. This is useful for finding the local time in a different offset. For example, if this time represents `10:30+02:00` and the offset specified is `+03:00`, then this method will return `11:30+03:00`.

To change the offset without adjusting the local time use `withOffsetSameLocal(java.time.ZoneOffset)`.

This instance is immutable and unaffected by this method call.

Parameters:

`offset - the zone offset to change to, not null`

Returns:

`an OffsetTime based on this time with the requested offset, not null`

toLocalTime

```
public LocalTime toLocalTime()
```

Gets the `LocalTime` part of this date-time.

This returns a `LocalTime` with the same hour, minute, second and nanosecond as this date-time.

Returns:

`the time part of this date-time, not null`

getHour

```
public int getHour()
```

Gets the hour-of-day field.

Returns:

`the hour-of-day, from 0 to 23`

getMinute

```
public int getMinute()
```

Gets the minute-of-hour field.

Returns:

the minute-of-hour, from 0 to 59

getSecond

```
public int getSecond()
```

Gets the second-of-minute field.

Returns:

the second-of-minute, from 0 to 59

getNano

```
public int getNano()
```

Gets the nano-of-second field.

Returns:

the nano-of-second, from 0 to 999,999,999

with

```
public OffsetTime with(TemporalAdjuster adjuster)
```

Returns an adjusted copy of this time.

This returns an `OffsetTime`, based on this one, with the time adjusted. The adjustment takes place using the specified adjuster strategy object. Read the documentation of the adjuster to understand what adjustment will be made.

A simple adjuster might simply set the one of the fields, such as the hour field. A more complex adjuster might set the time to the last hour of the day.

The classes `LocalTime` and `ZoneOffset` implement `TemporalAdjuster`, thus this method can be used to change the time or offset:

```
  result = offsetTime.with(time);
  result = offsetTime.with(offset);
```

The result of this method is obtained by invoking the `TemporalAdjuster.adjustInto(Temporal)` method on the specified adjuster passing `this` as the argument.

This instance is immutable and unaffected by this method call.

Specified by:

with in interface Temporal

Parameters:

adjuster - the adjuster to use, not null

Returns:

an OffsetTime based on this with the adjustment made, not null

Throws:

DateTimeException - if the adjustment cannot be made

ArithmeticException - if numeric overflow occurs

with

```
public OffsetTime with(TemporalField field,
                       long newValue)
```

Returns a copy of this time with the specified field set to a new value.

This returns an OffsetTime, based on this one, with the value for the specified field changed. This can be used to change any supported field, such as the hour, minute or second. If it is not possible to set the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a ChronoField then the adjustment is implemented here.

The OFFSET_SECONDS field will return a time with the specified offset. The local time is unaltered. If the new offset value is outside the valid range then a DateTimeException will be thrown.

The other supported fields will behave as per the matching method on LocalTime.with(TemporalField, long) LocalTime}. In this case, the offset is not part of the calculation and will be unchanged.

All other ChronoField instances will throw an UnsupportedTemporalTypeException.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.adjustInto(Temporal, long) passing this as the argument. In this case, the field determines whether and how to adjust the instant.

This instance is immutable and unaffected by this method call.

Specified by:

with in interface Temporal

Parameters:

field - the field to set in the result, not null

newValue - the new value of the field in the result

Returns:

an OffsetTime based on this with the specified field set, not null

Throws:

DateTimeException - if the field cannot be set

UnsupportedTemporalTypeException - if the field is not supported

ArithmeticException - if numeric overflow occurs

withHour

public OffsetTime withHour(int hour)

Returns a copy of this OffsetTime with the hour-of-day altered.

The offset does not affect the calculation and will be the same in the result.

This instance is immutable and unaffected by this method call.

Parameters:

hour - the hour-of-day to set in the result, from 0 to 23

Returns:

an OffsetTime based on this time with the requested hour, not null

Throws:

DateTimeException - if the hour value is invalid

withMinute

public OffsetTime withMinute(int minute)

Returns a copy of this OffsetTime with the minute-of-hour altered.

The offset does not affect the calculation and will be the same in the result.

This instance is immutable and unaffected by this method call.

Parameters:

minute - the minute-of-hour to set in the result, from 0 to 59

Returns:

an OffsetTime based on this time with the requested minute, not null

Throws:

DateTimeException - if the minute value is invalid

withSecond

```
public OffsetTime withSecond(int second)
```

Returns a copy of this `OffsetTime` with the second-of-minute altered.

The offset does not affect the calculation and will be the same in the result.

This instance is immutable and unaffected by this method call.

Parameters:

`second - the second-of-minute to set in the result, from 0 to 59`

Returns:

`an OffsetTime based on this time with the requested second, not null`

Throws:

`DateTimeException - if the second value is invalid`

withNano

```
public OffsetTime withNano(int nanoOfSecond)
```

Returns a copy of this `OffsetTime` with the nano-of-second altered.

The offset does not affect the calculation and will be the same in the result.

This instance is immutable and unaffected by this method call.

Parameters:

`nanoOfSecond - the nano-of-second to set in the result, from 0 to 999,999,999`

Returns:

`an OffsetTime based on this time with the requested nanosecond, not null`

Throws:

`DateTimeException - if the nanos value is invalid`

truncatedTo

```
public OffsetTime truncatedTo(TemporalUnit unit)
```

Returns a copy of this `OffsetTime` with the time truncated.

Truncation returns a copy of the original time with fields smaller than the specified unit set to zero. For example, truncating with the `minutes` unit will set the second-of-minute and nano-of-second field to zero.

The unit must have a duration that divides into the length of a standard day without remainder. This includes all supplied time units on `ChronoUnit` and `DAYS`. Other units throw an exception.

The offset does not affect the calculation and will be the same in the result.

This instance is immutable and unaffected by this method call.

Parameters:

`unit - the unit to truncate to, not null`

Returns:

`an OffsetTime based on this time with the time truncated, not null`

Throws:

`DateTimeException - if unable to truncate`

`UnsupportedTemporalTypeException - if the unit is not supported`

plus

```
public OffsetTime plus(TemporalAmount amountToAdd)
```

Returns a copy of this time with the specified amount added.

This returns an `OffsetTime`, based on this one, with the specified amount added. The amount is typically `Duration` but may be any other type implementing the `TemporalAmount` interface.

The calculation is delegated to the amount object by calling `TemporalAmount.addTo(Temporal)`. The amount implementation is free to implement the addition in any way it wishes, however it typically calls back to `plus(long, TemporalUnit)`. Consult the documentation of the amount implementation to determine if it can be successfully added.

This instance is immutable and unaffected by this method call.

Specified by:

`plus in interface Temporal`

Parameters:

`amountToAdd - the amount to add, not null`

Returns:

`an OffsetTime based on this time with the addition made, not null`

Throws:

`DateTimeException - if the addition cannot be made`

ArithmeticException - if numeric overflow occurs

plus

```
public OffsetTime plus(long amountToAdd,
                       TemporalUnit unit)
```

Returns a copy of this time with the specified amount added.

This returns an `OffsetTime`, based on this one, with the amount in terms of the unit added. If it is not possible to add the amount, because the unit is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoUnit` then the addition is implemented by `LocalTime.plus(long, TemporalUnit)`. The offset is not part of the calculation and will be unchanged in the result.

If the field is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.addTo(Temporal, long)` passing `this` as the argument. In this case, the unit determines whether and how to perform the addition.

This instance is immutable and unaffected by this method call.

Specified by:

`plus` in interface `Temporal`

Parameters:

`amountToAdd` - the amount of the unit to add to the result, may be negative

`unit` - the unit of the amount to add, not null

Returns:

an OffsetTime based on this time with the specified amount added, not null

Throws:

`DateTimeException` - if the addition cannot be made

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

plusHours

```
public OffsetTime plusHours(long hours)
```

Returns a copy of this `OffsetTime` with the specified number of hours added.

This adds the specified number of hours to this time, returning a new time. The calculation wraps around midnight.

This instance is immutable and unaffected by this method call.

Parameters:

`hours - the hours to add, may be negative`

Returns:

`an OffsetTime based on this time with the hours added, not null`

plusMinutes

`public OffsetTime plusMinutes(long minutes)`

Returns a copy of this `OffsetTime` with the specified number of minutes added.

This adds the specified number of minutes to this time, returning a new time. The calculation wraps around midnight.

This instance is immutable and unaffected by this method call.

Parameters:

`minutes - the minutes to add, may be negative`

Returns:

`an OffsetTime based on this time with the minutes added, not null`

plusSeconds

`public OffsetTime plusSeconds(long seconds)`

Returns a copy of this `OffsetTime` with the specified number of seconds added.

This adds the specified number of seconds to this time, returning a new time. The calculation wraps around midnight.

This instance is immutable and unaffected by this method call.

Parameters:

`seconds - the seconds to add, may be negative`

Returns:

`an OffsetTime based on this time with the seconds added, not null`

plusNanos

```
public OffsetTime plusNanos(long nanos)
```

Returns a copy of this `OffsetTime` with the specified number of nanoseconds added.

This adds the specified number of nanoseconds to this time, returning a new time. The calculation wraps around midnight.

This instance is immutable and unaffected by this method call.

Parameters:

`nanos - the nanos to add, may be negative`

Returns:

`an OffsetTime based on this time with the nanoseconds added, not null`

minus

```
public OffsetTime minus(TemporalAmount amountToSubtract)
```

Returns a copy of this time with the specified amount subtracted.

This returns an `OffsetTime`, based on this one, with the specified amount subtracted. The amount is typically `Duration` but may be any other type implementing the `TemporalAmount` interface.

The calculation is delegated to the amount object by calling `TemporalAmount.subtractFrom(Temporal)`. The amount implementation is free to implement the subtraction in any way it wishes, however it typically calls back to `minus(long, TemporalUnit)`. Consult the documentation of the amount implementation to determine if it can be successfully subtracted.

This instance is immutable and unaffected by this method call.

Specified by:

`minus` in interface `Temporal`

Parameters:

`amountToSubtract` - the amount to subtract, not null

Returns:

an OffsetTime based on this time with the subtraction made, not null

Throws:

`DateTimeException` - if the subtraction cannot be made

`ArithmeticException` - if numeric overflow occurs

minus

```
public OffsetTime minus(long amountToSubtract,
                        TemporalUnit unit)
```

Returns a copy of this time with the specified amount subtracted.

This returns an `OffsetTime`, based on this one, with the amount in terms of the unit subtracted. If it is not possible to subtract the amount, because the unit is not supported or for some other reason, an exception is thrown.

This method is equivalent to `plus(long, TemporalUnit)` with the amount negated. See that method for a full description of how addition, and thus subtraction, works.

This instance is immutable and unaffected by this method call.

Specified by:

minus in interface Temporal

Parameters:

amountToSubtract - the amount of the unit to subtract from the result, may be negative

unit - the unit of the amount to subtract, not null

Returns:

an OffsetTime based on this time with the specified amount subtracted, not null

Throws:

DateTimeException - if the subtraction cannot be made

UnsupportedTemporalTypeException - if the unit is not supported

ArithmeticException - if numeric overflow occurs

minusHours

public OffsetTime minusHours(long hours)

Returns a copy of this OffsetTime with the specified number of hours subtracted.

This subtracts the specified number of hours from this time, returning a new time. The calculation wraps around midnight.

This instance is immutable and unaffected by this method call.

Parameters:

hours - the hours to subtract, may be negative

Returns:

an OffsetTime based on this time with the hours subtracted, not null

minusMinutes

public OffsetTime minusMinutes(long minutes)

Returns a copy of this OffsetTime with the specified number of minutes subtracted.

This subtracts the specified number of minutes from this time, returning a new time. The calculation wraps around midnight.

This instance is immutable and unaffected by this method call.

Parameters:

minutes - the minutes to subtract, may be negative

Returns:

an OffsetTime based on this time with the minutes subtracted, not null

minusSeconds

```
public OffsetTime minusSeconds(long seconds)
```

Returns a copy of this OffsetTime with the specified number of seconds subtracted.

This subtracts the specified number of seconds from this time, returning a new time. The calculation wraps around midnight.

This instance is immutable and unaffected by this method call.

Parameters:

seconds - the seconds to subtract, may be negative

Returns:

an OffsetTime based on this time with the seconds subtracted, not null

minusNanos

```
public OffsetTime minusNanos(long nanos)
```

Returns a copy of this OffsetTime with the specified number of nanoseconds subtracted.

This subtracts the specified number of nanoseconds from this time, returning a new time. The calculation wraps around midnight.

This instance is immutable and unaffected by this method call.

Parameters:

nanos - the nanos to subtract, may be negative

Returns:

an OffsetTime based on this time with the nanoseconds subtracted, not null

query

```
public <R> R query(TemporalQuery<R> query)
```

Queries this time using the specified query.

This queries this time using the specified query strategy object. The `TemporalQuery` object defines the logic to be used to obtain the result. Read the documentation of the query to understand what the result of this method will be.

The result of this method is obtained by invoking the `TemporalQuery.queryFrom(TemporalAccessor)` method on the specified query passing `this` as the argument.

Specified by:

`query` in interface `TemporalAccessor`

Type Parameters:

`R` - the type of the result

Parameters:

`query` - the query to invoke, not null

Returns:

the query result, null may be returned (defined by the query)

Throws:

`DateTimeException` - if unable to query (defined by the query)

`ArithmeticException` - if numeric overflow occurs (defined by the query)

adjustInto

```
public Temporal adjustInto(Temporal temporal)
```

Adjusts the specified temporal object to have the same offset and time as this object.

This returns a temporal object of the same observable type as the input with the offset and time changed to be the same as this.

The adjustment is equivalent to using `Temporal.with(TemporalField, long)` twice, passing `ChronoField.NANO_OF_DAY` and `ChronoField.OFFSET_SECONDS` as the fields.

In most cases, it is clearer to reverse the calling pattern by using `Temporal.with(TemporalAdjuster)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisOffsetTime.adjustInto(temporal);
temporal = temporal.with(thisOffsetTime);
```

This instance is immutable and unaffected by this method call.

Specified by:

`adjustInto` in interface `TemporalAdjuster`

Parameters:

`temporal` - the target object to be adjusted, not null

Returns:

the adjusted object, not null

Throws:

`DateTimeException` - if unable to make the adjustment

`ArithmeticException` - if numeric overflow occurs

until

```
public long until(Temporal endExclusive,
                  TemporalUnit unit)
```

Calculates the amount of time until another time in terms of the specified unit.

This calculates the amount of time between two `OffsetTime` objects in terms of a single `TemporalUnit`. The start and end points are `this` and the specified time. The result will be negative if the end is before the start. For example, the amount in hours between two times can be calculated using `startTime.until(endTime, HOURS)`.

The `Temporal` passed to this method is converted to a `OffsetTime` using `from(TemporalAccessor)`. If the offset differs between the two times, then the specified end time is normalized to have the same offset as this time.

The calculation returns a whole number, representing the number of complete units between the two times. For example, the amount in hours between 11:30Z and 13:29Z will only be one hour as it is one minute short of two hours.

There are two equivalent ways of using this method. The first is to invoke this method. The second is to use `TemporalUnit.between(Temporal, Temporal)`:

```
// these two lines are equivalent
amount = start.until(end, MINUTES);
amount = MINUTES.between(start, end);
```

The choice should be made based on which makes the code more readable.

The calculation is implemented in this method for `ChronoUnit`. The units `NANOS`, `MICROS`, `MILLIS`, `SECONDS`, `MINUTES`, `HOURS` and `HALF_DAYS` are supported. Other `ChronoUnit` values will throw an exception.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.between(Temporal, Temporal)` passing `this` as the first argument and the converted input temporal as the second argument.

This instance is immutable and unaffected by this method call.

Specified by:

`until` in interface `Temporal`

Parameters:

`endExclusive - the end time, exclusive, which is converted to an OffsetTime, not null`

`unit - the unit to measure the amount in, not null`

Returns:

`the amount of time between this time and the end time`

Throws:

`DateTimeException - if the amount cannot be calculated, or the end temporal cannot be converted to an OffsetTime`

`UnsupportedTemporalTypeException - if the unit is not supported`

`ArithmeticException - if numeric overflow occurs`

format

```
public String format(DateTimeFormatter formatter)
```

Formats this time using the specified formatter.

This time will be passed to the formatter to produce a string.

Parameters:

`formatter - the formatter to use, not null`

Returns:

`the formatted time string, not null`

Throws:

`DateTimeException - if an error occurs during printing`

atDate

```
public OffsetDateTime atDate(LocalDate date)
```

Combines this time with a date to create an `OffsetDateTime`.

This returns an `OffsetDateTime` formed from this time and the specified date. All possible combinations of date and time are valid.

Parameters:

`date - the date to combine with, not null`

Returns:

`the offset date-time formed from this time and the specified date, not null`

compareTo

```
public int compareTo(OffsetTime other)
```

Compares this `OffsetTime` to another time.

The comparison is based first on the UTC equivalent instant, then on the local time. It is "consistent with equals", as defined by `Comparable`.

For example, the following is the comparator order:

1. `10:30+01:00`
2. `11:00+01:00`
3. `12:00+02:00`
4. `11:30+01:00`
5. `12:00+01:00`
6. `12:30+01:00`

Values #2 and #3 represent the same instant on the time-line. When two values represent the same instant, the local time is compared to distinguish them. This step is needed to make the ordering consistent with `equals()`.

To compare the underlying local time of two `TemporalAccessor` instances, use `ChronoField.NANO_OF_DAY` as a comparator.

Specified by:

`compareTo in interface Comparable<OffsetTime>`

Parameters:

`other - the other time to compare to, not null`

Returns:

`the comparator value, negative if less, positive if greater`

Throws:

`NullPointerException - if other is null`

isAfter

```
public boolean isAfter(OffsetTime other)
```

Checks if the instant of this `OffsetTime` is after that of the specified time applying both times to a common date.

This method differs from the comparison in compareTo(java.time.OffsetTime) in that it only compares the instant of the time. This is equivalent to converting both times to an instant using the same date and comparing the instants.

Parameters:

other - the other time to compare to, not null

Returns:

true if this is after the instant of the specified time

isBefore

```
public boolean isBefore(OffsetTime other)
```

Checks if the instant of this OffsetTime is before that of the specified time applying both times to a common date.

This method differs from the comparison in compareTo(java.time.OffsetTime) in that it only compares the instant of the time. This is equivalent to converting both times to an instant using the same date and comparing the instants.

Parameters:

other - the other time to compare to, not null

Returns:

true if this is before the instant of the specified time

isEqual

```
public boolean isEqual(OffsetTime other)
```

Checks if the instant of this OffsetTime is equal to that of the specified time applying both times to a common date.

This method differs from the comparison in compareTo(java.time.OffsetTime) and equals(java.lang.Object) in that it only compares the instant of the time. This is equivalent to converting both times to an instant using the same date and comparing the instants.

Parameters:

other - the other time to compare to, not null

Returns:

true if this is equal to the instant of the specified time

equals

```
public boolean equals(Object obj)
```

Checks if this time is equal to another time.

The comparison is based on the local-time and the offset. To compare for the same instant on the time-line, use `isEqual(OffsetTime)`.

Only objects of type `OffsetTime` are compared, other types return false. To compare the underlying local time of two `TemporalAccessor` instances, use `ChronoField.NANO_OF_DAY` as a comparator.

Overrides:

`equals` in class `Object`

Parameters:

`obj - the object to check, null returns false`

Returns:

`true if this is equal to the other time`

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

A hash code for this time.

Overrides:

`hashCode` in class `Object`

Returns:

`a suitable hash code`

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(java.lang.Object)`

toString

```
public String toString()
```

Outputs this time as a `String`, such as `10:15:30+01:00`.

The output will be one of the following ISO-8601 formats:

- `HH:mmXXXXX`

- HH:mm:ssXXXXX
- HH:mm:ss.SSSXXXXX
- HH:mm:ss.SSSSSSXXXXX
- HH:mm:ss.SSSSSSSSSXXXXX

The format used will be the shortest that outputs the full value of the time where the omitted parts are implied to be zero.

Overrides:

`toString` in class `Object`

Returns:

a string representation of this time, not null

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.